

Case Study Report on

“RecyclerView List Display Application”

*project submitted in fulfilment of the requirements for the
award of the degree of*

BACHELOR OF TECHNOLOGY IN CSE (AI & ML)

Submitted by

Ch.GanaSurya	A23126552277
M.VenkatTeja	A23126552278
A.Chandu	A23126552279
K.kulwanth Hari Sai	A23126552280
K.L.S.S.Abhi Ram	A23126552281

Under the Guidance of

Mr. P. Santosh Kumar

(Assistant Professor)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA

& NAAC with ‘A+’ Grade)

SANGIVALA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162

2025 – 2026

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

**(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA
& NAAC with 'A+' Grade)**

SANGIVALA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162



BONAFIDE CERTIFICATE

This is to certify that the Mini Project entitled “RecyclerView List Display Application” submitted by Team 8 in partial fulfilment of the requirements for the award of Bachelor of Technology in CSE (AI & ML) in the Department of Computer Science and Engineering (AI & ML), Anil Neerukonda Institute of Technology and Sciences (A+), Visakhapatnam, is a record of bonafide work carried out under my guidance and supervision.

Mr. P. Santosh Kumar

ASSISTANT PROFESSOR

Department of CSE (AI & ML)

ACKNOWLEDGEMENT

This case study reflects our academic learning and professional development journey. The case study provided an opportunity to apply classroom knowledge in a real-world cloud and containerization environment.

We express sincere gratitude to Dr. K. Selvani Deepthi, Head of the Department of Computer Science and Engineering(AI&ML) at ANITS, and our guide Mr. Santosh Kumar P, Associate Professor, for invaluable guidance and encouragement throughout the case study. We also acknowledge the faculty members for their unwavering support.

Ch.GanaSurya	A23126552277
M.VenkatTeja	A23126552278
A.Chandu	A23126552279
K.kulwanth Hari Sai	A23126552280
K.L.S.S.Abhi Ram	A23126552281

CONTENTS

Sl. No.	Title	Page No.
1	Abstract	5
2	Introduction	5
3	Aim	6
4	Objective	6
5	Tools Used	6
6	Architecture Diagram	7
7	Procedure	8
8	Implementation	8
9	Output	11
10	Conclusion	13
11	Advantages	13
12	Limitations	13
13	References	14

ABSTRACT

This project presents the design and development of an Android application that demonstrates the efficient display of data using RecyclerView. In modern mobile applications, handling large datasets in a smooth and optimized way is a crucial requirement. RecyclerView is a powerful and flexible UI component provided by Android that allows developers to display large amounts of data efficiently by reusing views and minimizing memory consumption.

The application developed in this project displays a list of items such as fruits in a scrollable format. The implementation uses important Android concepts such as Adapter, ViewHolder pattern, and LayoutManager. These components work together to ensure that only the visible items are rendered on the screen, thereby improving performance and reducing resource usage.

The application is developed using Kotlin as the programming language and XML for designing the user interface. The project focuses on providing a clean and responsive UI along with smooth scrolling behavior. It demonstrates how RecyclerView can be used in real-world applications such as contact lists, product listings, and social media feeds.

Overall, this project provides practical knowledge of Android development and highlights the importance of efficient UI design and data handling techniques. It serves as a strong foundation for building advanced Android applications.

INTRODUCTION

Mobile applications have become an essential part of everyday life, enabling users to perform various tasks efficiently and conveniently. One of the key aspects of mobile application development is presenting data in a structured and user-friendly manner. As applications grow more complex, the need for efficient data handling and display becomes increasingly important.

RecyclerView is an advanced UI component introduced in Android to overcome the limitations of ListView and GridView. It provides a more flexible and efficient way of displaying large datasets. Unlike traditional views, RecyclerView uses a recycling mechanism that reuses existing views instead of creating new ones repeatedly. This significantly improves performance, especially when dealing with large amounts of data.

In this project, an Android application is developed to demonstrate the use of RecyclerView for displaying a list of items. The application shows a scrollable list of fruits, allowing users to navigate through the data smoothly. The implementation involves creating a custom Adapter class, defining a ViewHolder, and using a LayoutManager to control the layout of the list. The project is developed using Android Studio, which provides a complete environment for building Android applications. Kotlin is used as the programming language due to its simplicity and modern features, while XML is used for designing the user interface. This project helps in understanding core Android development concepts such as UI design, event handling, and efficient data management. It also provides insight into how modern applications handle large datasets efficiently.

AIM

The main aim of this project is to design and develop an Android application that efficiently displays a list of items using RecyclerView, ensuring smooth scrolling, optimized performance, and user-friendly interface design..

OBJECTIVE

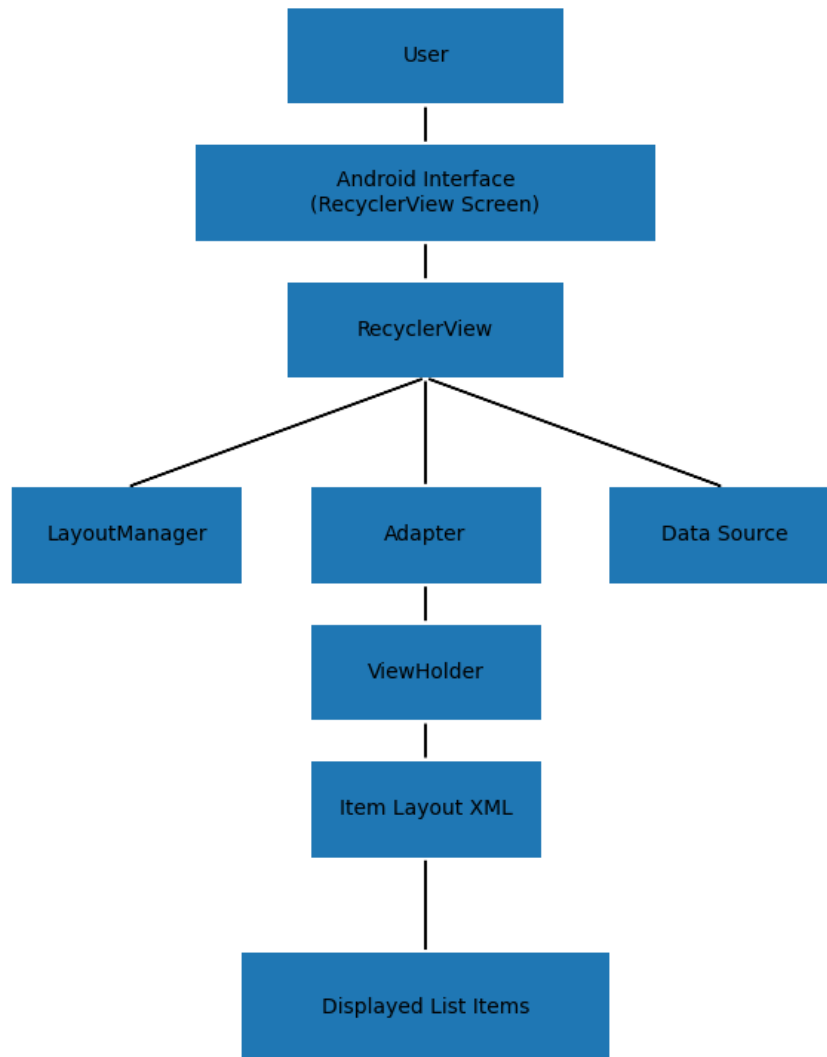
The objectives of this project are:

1. To understand the concept and importance of RecyclerView in Android development
2. To design a user interface using XML layouts
3. To implement a custom Adapter for binding data to views
4. To use the ViewHolder pattern for efficient memory usage
5. To implement LayoutManager for organizing list items
6. To display dynamic data in a scrollable list format
7. To improve performance using view recycling techniques
8. To gain hands-on experience in Android app development
9. To create a simple and interactive user interface
10. To understand real-world applications of RecyclerView

TOOLS USED

Tool / Technology	Description
Android Studio	Integrated Development Environment used to build and run the Android application
Kotlin	Used for writing the application logic and handling interactions
XML	Used to design layouts, UI components, and resources
Material Design	Provides UI components and styling guidelines for modern app design
Android SDK 34	Platform used to develop and test the application on Android devices

ARCHITECTURE DIAGRAM



ARCHITECTURE EXPLANATION

The application follows a simple and modular architecture. The user interacts with the RecyclerView interface, which displays a list of items. The RecyclerView acts as a container for displaying data efficiently.

The Adapter serves as a bridge between the data source and the RecyclerView. It takes the data and binds it to the individual views. The ViewHolder pattern is used to store references to views, which reduces the need for repeated view lookups and improves performance.

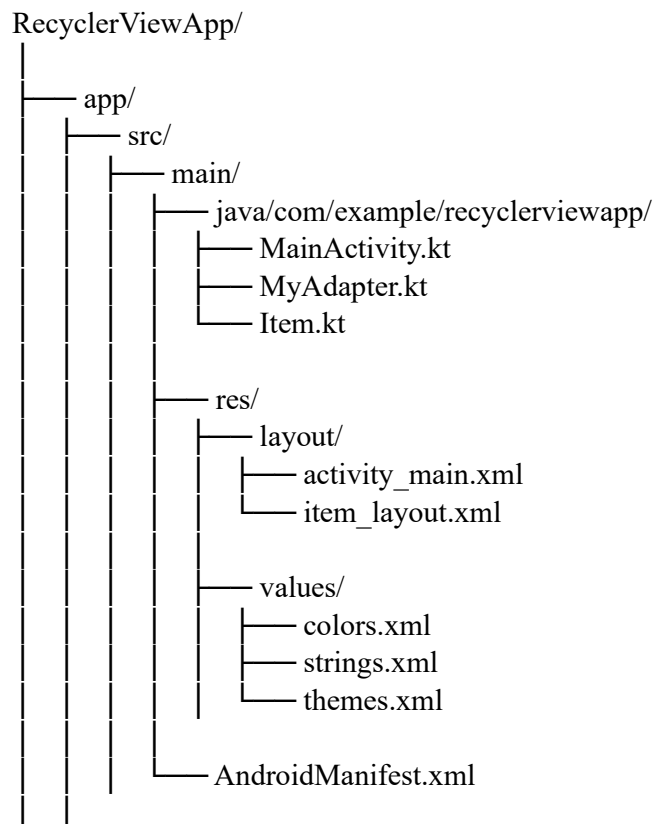
The LayoutManager controls how items are arranged on the screen. In this project, LinearLayoutManager is used to display items in a vertical list.

The data source contains the list of items that are displayed in the RecyclerView. The interaction between these components ensures smooth scrolling and efficient data handling.

PROCEDURE

- Install Android Studio and set up the environment
- Create a new project using Kotlin
- Select Empty Activity template
- Configure project details such as name and SDK version
- Add RecyclerView dependency in Gradle file
- Design activity_main.xml with RecyclerView
- Create item_layout.xml for list items
- Define a data class (Item.kt)
- Create Adapter class (MyAdapter.kt)
- Implement ViewHolder pattern Bind data in Adapter
- Set LayoutManager in MainActivity
- Attach Adapter to RecyclerView
- Run the application on emulator/device
- Test scrolling and UI behavior.

Project Folder Structure



| └─ build.gradle

IMPLEMENTATION

The implementation of the project involves multiple components working together.

MainActivity

This is the main entry point of the application. It initializes the RecyclerView, creates the data list, and sets the Adapter and LayoutManager.

) MainActivity.kt

```
package com.example.recyclerviewapp
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val recyclerView = findViewById<RecyclerView>(R.id.recyclerView)
        val itemList = listOf(
            Item("Apple"),
            Item("Banana"),
            Item("Mango"),
            Item("Orange"),
            Item("Grapes"),
            Item("Pineapple"),
            Item("Papaya"),
            Item("Strawberry"),
            Item("Kiwi"),
            Item("Guava"),
            Item("Cherry"),
            Item("Blueberry"),
            Item("Peach"),
            Item("Plum"),
            Item("Pear"),
            Item("Lychee"),
            Item("Dragon Fruit"),
            Item("Pomegranate"),
            Item("Apricot"),
            Item("Fig"),
            Item("Coconut"),
            Item("Blackberry"),
        )
```

```

        Item("Raspberry"),
        Item("Passion Fruit"),
        Item("Jackfruit")
    )

    recyclerView.layoutManager = LinearLayoutManager(this)
    recyclerView.adapter = MyAdapter(itemList)
}

```

Adapter

The Adapter binds the data to the views. It creates ViewHolder objects and binds data to each item.

```
package com.example.recyclerviewapp
```

```

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class MyAdapter(private val itemList: List<Item>) :
    RecyclerView.Adapter<MyAdapter.ViewHolder>() {

    // ViewHolder class
    class ViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val textView: TextView = itemView.findViewById(R.id.textView)
    }

    // Called when a new ViewHolder is created
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_layout, parent, false)
        return ViewHolder(view)
    }

    // Binds data to the ViewHolder
    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        holder.textView.text = itemList[position].name
    }

    // Returns total number of items
    override fun getItemCount(): Int {
        return itemList.size
    }
}

```

Data Class

The data class represents each item in the list.

XML Layouts

XML files define the UI structure of the application.

XML Layout (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.recyclerview.widget.RecyclerView
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/recyclerView"    android:layout_width="match_parent"
android:layout_height="match_parent"/>
```

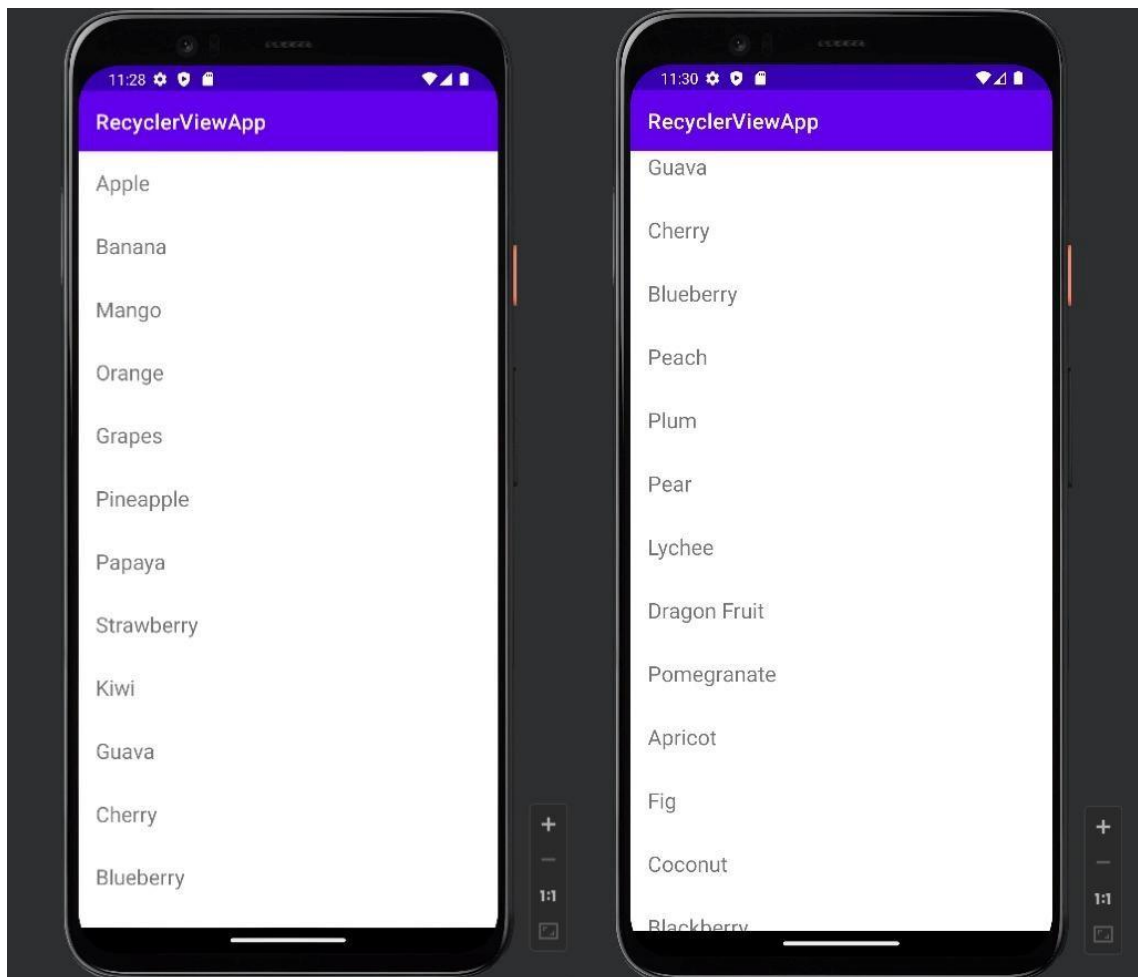
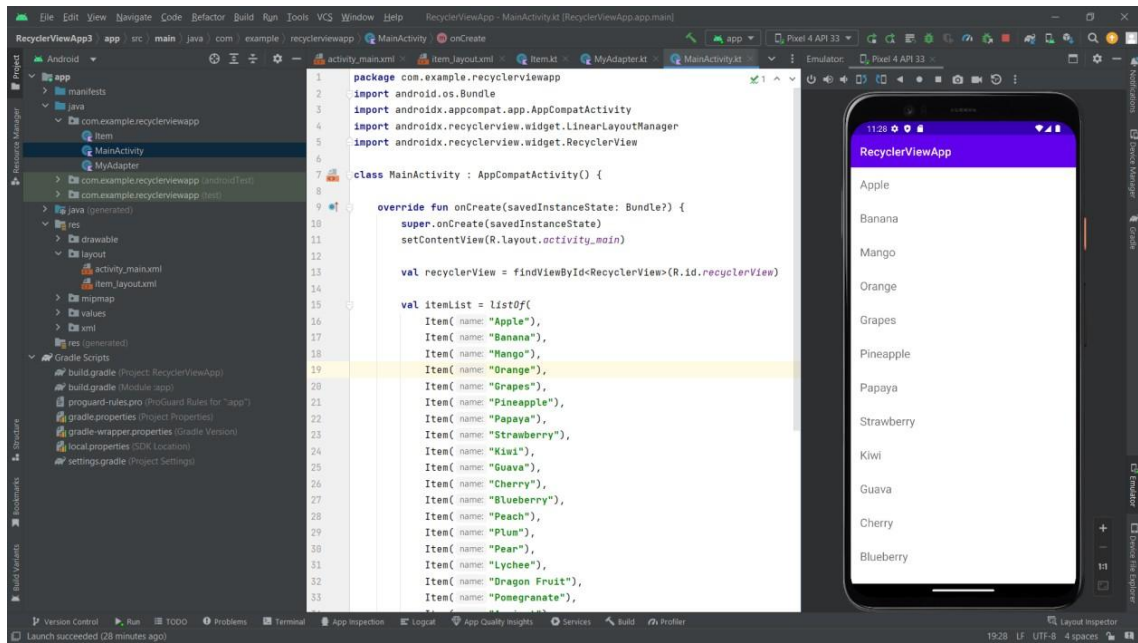
Item Layout (item_layout.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<TextView
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/textView"
android:layout_width="match_parent"
android:layout_height="wrap_content"    android:textSize="20sp"
android:padding="16dp"/>
```

OUTPUT

After running the application:

- The welcome title is displayed.
- Messages rotate automatically.
- Clicking the button changes the message and button text.
- Clicking the card shows or hides detailed AI & ML information.
- Statistics are displayed at the bottom.



RESULTS

The application was successfully implemented and tested. The RecyclerView efficiently displays a large list of items with smooth scrolling. The Adapter correctly binds data, and the ViewHolder pattern ensures optimized performance.

The application runs without errors and provides a good user experience. It demonstrates the advantages of using RecyclerView over traditional list components.

CONCLUSION

The RecyclerView List Display Application successfully demonstrates how modern Android applications handle large datasets efficiently. By using RecyclerView, Adapter, and ViewHolder pattern, the application achieves high performance and smooth scrolling.

This project provides practical knowledge of Android development and helps in understanding important UI components. It also highlights the importance of efficient data handling and optimized UI design.

The knowledge gained from this project can be applied to develop real-world applications such as e-commerce apps, messaging platforms, and dashboards.

ADVANTAGES

- Efficient memory usage
- Smooth scrolling performance
- Reusable components
- Flexible and customizable
- Suitable for large datasets
- Better performance than ListView
- Supports multiple view types

LIMITATIONS

- Requires more code compared to ListView
- No backend integration
- Static data used
- Limited UI design in this project
- Requires understanding of Adapter pattern
- Not suitable for very simple lists

REFERENCES

1. Android Developers. Android Developer Documentation. Available: <https://developer.android.com>
2. Android Developers. RecyclerView Guide. Available: <https://developer.android.com/guide/topics/ui/layout/recyclerview>
3. JetBrains. Kotlin Programming Language Documentation. Available: <https://kotlinlang.org>
4. Google. Material Design Guidelines. Available: <https://material.io>
5. Android Developers. UI Components and Layouts. Available: <https://developer.android.com/guide/topics/ui>
6. GitHub. RecyclerView Sample Projects. Available: <https://github.com>
7. Android Developers. Activity Lifecycle Documentation. Available: <https://developer.android.com/guide/components/activities/activity-lifecycle>